

Ejercicios de exámenes

1. Tenemos un fichero de texto llamado personajes.txt con la siguiente estructura:

```
4 Tanjiro Kamado
17 Naruto Uzumaki
3 Vinsmoke Sanji
22 Ken Takakura
5 Shinji Ikari
3 Roronoa Zoro
```

Y otro llamado animes.txt con la siguiente:

```
17 Naruto
22 Dan Da Dan
4 Demon Slayer
3 One Piece
6 Hunter x Hunter
```

En el primero aparecen personajes de Anime con un número. En el segundo el título del anime con otro número. El número puede ser cualquier entero y actúa como una clave: los personajes con un determinado número son los relacionados con el anime que tiene el mismo número.

Como ves, pueden aparecer personajes para los que no hay anime y animes que no tienen personaje. Los números no tienen por qué presentar un orden lógico. También puede haber un anime con más de un personaje. Por lo demás, el formato del fichero se considera siempre correcto.

Escribe una función en Java que lea ambos ficheros y nos muestre una salida como la que sigue por consola:

```
One Piece
- Roronoa Zoro
- Vinsmoke Sanji

Demon Slayer
- Tanjiro Kamado

Hunter x Hunter
- No hay personajes

Naruto
- Naruto Uzumaki

Dan Da Dan
- Ken Takakura
```

Personajes sin anime **- Shinji Ikari**

Si no se te ocurre la estrategia a seguir, te sugiero la siguiente:

- ◆ En primer lugar lee el fichero de animes y crea un diccionario que tenga por clave el número que lo identifica y por valor el título
- ◆ Luego, por cada entrada del diccionario, abre el fichero de personajes y recórrelo buscando personajes con el mismo código de identificación y construye una lista con ellos. Si lo haces de forma modular (una función que recibe un código y el nombre del fichero y devuelve la lista) es muy sencillo
- ◆ Te faltaría, por último, los personajes que no tienen anime. Recorre por última vez el fichero de personajes y crea una lista con todos aquellos personajes cuyo código no tenga una entrada en el fichero de animes.

2. Crea ahora una clase llamada **Personaje** que tenga como atributos el título del anime y el nombre del personaje. Luego escribe una función que te escriba en un fichero binario llamado **personajes.dat** todos los objetos que estén completos (es decir, obviando los personajes sin anime y los animes sin personaje) y otra que te lea del fichero anterior y te muestre la información en pantalla de la siguiente forma:

Roronoa Zoro (One Piece)
Vinsmoke Sanji (One Piece)
Tanjiro Kamado (Demon Slayer)
Naruto Uzumaki (Naruto)
Ken Takakura (Dan Da Dan)

IMPORTANTE: Si no has hecho o no has hecho correctamente el primer ejercicio, puedes invocar al constructor de la clase directamente desde código escribiendo los datos y sin leer el fichero para que pueda valorarte la parte de escritura y lectura en archivos binarios. Lógicamente, no puntuará lo mismo si decides resolverlo de esta forma.

NOTAS:

- ◆ El segundo ejercicio tiene muchos elementos comunes con el primero. Puedes reutilizar todo el código que quieras entre ambos.
- ◆ Se valorará, como siempre, la modularidad de la solución, que no se repita código innecesariamente, etc.
- ◆ Lógicamente, tus funciones deberían de funcionar con ficheros con otro contenido y no solo con los que aquí aparecen pero siempre con la misma estructura aquí descrita, claro.

3. Tenemos un fichero de texto llamado `países.csv` con la siguiente estructura:

País,Capital,Moneda,Animal
Australia,Canberra,Dólar australiano,Canguro
Canadá,Ottawa,Dólar canadiense,Castor
Madagascar,Antananarivo,Ariary malgache,Lémur
China,Pekín,Renminbi (Yuan),Oso panda gigante
Perú,Lima,Sol,Vicuña
Nueva Zelanda,Wellington,Dólar neozelandés,Kiwi

Escribe un programa en Java que lea de ese fichero y nos muestre la información del mismo de la siguiente forma:

Países en el fichero: 6

Nombres: Australia, Canadá, Madagascar, China, Perú y Nueva Zelanda

Las capitales de los mismos son: Canberra, Ottawa, Antananarivo, Pekín, Lima y Wellington

Sus monedas oficiales son: Dolar australiano, Dolar canadiense, Ariary malgache, Renminbi (Yuan), Sol y Dolar neozelandes

Sus animales mas representativos son Canguro, Castor, Lémur, Oso panda gigante, Vicuña y Kiwi

NOTAS:

- Nuestro cliente es muy “especialito” con el idioma y quiere que no olvides poner esa letra “y” al final de los dos últimos elementos de cada lista. Piensa que el fichero podría tener los datos de un único país.
- Puede que haya líneas erróneas en cuanto al formato que ves ahí (4 datos separados por coma). Esas líneas deberían de ignorarse y no procesarse
- Fíjate bien que el separador es una coma y no una coma con un espacio en blanco a continuación
- La primera línea, si el fichero tiene contenido, siempre es la cabecera del fichero y debe de ignorarse también.
- El contenido del fichero puede variar con otros países, por supuesto. Tu programa debería de funcionar con cualquier otra lista o número de países. Incluido uno solo. O ninguno. En ese último caso mostrarías un mensaje apropiado:

No hay datos de ningún país en el fichero

4. Escribe un programa en Java que lea del fichero del ejercicio anterior y lo sobreescriba con el mismo contenido pero prescindiendo del nombre de la moneda. Por ejemplo, si leemos un fichero con el contenido del ejemplo mostrado antes debería de escribir lo siguiente:

País,Capital,Animal
Australia,Canberra,Canguro
Canadá,Ottawa,Castor
Madagascar,Antananarivo,Lémur
China,Pekín,Oso panda gigante
Perú,Lima,Vicuña
Nueva Zelanda,Wellington,Kiwi

NOTAS:

- Las consideraciones del ejercicio anterior son todas aplicables a este: el fichero podría tener otro contenido o estar vacío. La primera línea es de cabecera y no se toca. Las líneas erróneas (con una estructura diferente de 4 palabras separadas por coma) se ignoran y no se escriben en el fichero de nuevo, etc.
- El fichero que escribimos con el resultado debe de llamarse igual que el original (países.csv)

5. A las elecciones generales de un país imaginario concurren cuatro partidos políticos que llamaremos PA, PB, PC y PD. Te han encargado hacer el recuento de los datos y, para ello, recibirás los datos de las diferentes mesas electorales y tu tendrás que ir añadiendo esos votos a un fichero que se almacenará en binario por seguridad.

El fichero, incluirá por cada partido el nombre del mismo y el número de votos. El formato en el que lo grabes es cosa tuya (listas, objetos, datos individuales, etc.).

Los votos se añadirán a través de una función que tienes que hacer tú y que recibirá dos argumentos: el nombre del partido y el número de votos. Por ejemplo así:

anyadirVotos ("PC", 236)

La función abrirá el fichero, leerá su contenido, actualizará los votos del partido en cuestión (en este caso el partido PC), grabará el fichero con la nueva información y mostrará los nuevos resultados en consola. La salida será así:

Nuevos votos para el partido PC: 236.

Votos hasta el momento:

Partido PA: 134 votos

Partido PB: 98 votos

Partido PC: 456 votos

Partido PD: 177 votos

Lógicamente, la primera vez que llames a tu función el fichero deberá de crearse.

Por último, queremos que hagas otra función que nos permita ver los resultados. Esta función recibirá un número que representa el total del censo electoral para poder calcular el porcentaje del escrutinio. Así por ejemplo:

verResultados (1730)

Y la salida en consola sería así:

Resultados con un 50% de escrutinio:

Partido PC: 456 votos

Partido PD: 177 votos

Partido PA: 134 votos

Partido PB: 98 votos

Fíjate que en este caso los partidos aparecen ordenados por orden de votos. En caso de empate no importa cual se muestre primero.

6. En el equipo de seguridad de tu empresa quieren automatizar un cortafuegos básico. El programa leerá un historial de direcciones IP que han intentado acceder al servidor web en las últimas horas de un fichero y si una dirección IP aparece más de 2 veces en el historial, se considera un intento de ataque por fuerza bruta y debe ser bloqueada de inmediato

Escribe un programa en Java que pida por teclado el nombre del fichero donde se encuentra el registro de IPs. El programa deberá mostrar de forma ordenada la lista de las direcciones IP que han conseguido acceder limpiamente (las "permitidas" antes de su bloqueo) y, por otro lado, el listado de las IPs que han terminado bloqueadas junto al número total de intentos maliciosos que realizaron después de ser añadidas a la lista negra.

Si, por ejemplo, partimos de un fichero con este contenido:

```
192.168.1.1
10.0.0.5
192.168.1.1
172.16.15.15
192.168.1.1
10.0.0.5
192.168.1.1
```

La IP 192.168.1.1 aparece 4 veces. En las 2 primeras se le permite el acceso. A la tercera interacción es bloqueada de forma definitiva. Como tiene 4 apariciones ha registrado dos accesos superiores a lo permitido. Las dos restantes IPs que aparecen no incumplen la regla.

Tu salida en consola debería de ser así:

IP con acceso permitido:

- 10.0.0.5
- 172.16.15.15

IPs con acceso restringido:

- 192.168.1.1. Accesos superiores a lo permitido: 2

No tienes que realizar ningún tipo de validación sobre el contenido del fichero. Si tiene líneas son correctas, pero si tienes que contemplar que no exista o que no tenga contenido. En ambos casos deberías de mostrar un mensaje de advertencia de ello. Igualmente, si no hay IPs con acceso permitido o no aparecen IPs con acceso restringido esos bloques de texto no deberían de mostrarse en consola

7. La agencia espacial para la que trabajas ha interceptado un archivo binario secreto llamado `coordenadas.dat` que ya habrás descargado del aula virtual. Este archivo contiene la ubicación de un satélite perdido

La estructura del archivo es la siguiente:

- El archivo contiene una secuencia de registros. No sabemos cuantos son
- Cada registro ocupa exactamente 20 bytes con la siguiente estructura:
- ID del Satélite: Un número entero (int, 4 bytes).
- Latitud: Un número decimal (float, 4 bytes).
- Longitud: Un número decimal (float, 4 bytes).
- Código de Estado: Cuatro caracteres de texto (char a 2bytes cada uno, un total de 8bytes)

Te muestro a continuación el contenido de los dos primeros registros con el formato con el que debes de mostrarlos en pantalla:

Satélite ID: 101 | Posición: (40,4167, -3,7037) | Estado: ACTV
Satélite ID: 202 | Posición: (-22,9068, -43,1729) | Estado: WARN

Te han encargado que crees un programa en Java que lea de este archivo binario y muestre todos los registros que están guardados en él de la forma que se ha descrito.